Proyecto Intermodular 2

Sprint 4 – Proyecto 1

Instrucciones breves de ejecución



Tamara Morillo Gregorio 2º DAMS

ÍNDICE

1. Introducción 2
2. Entidad Examen 2
3. Repositorio de Exámenes 3
4. Controlador de Exámenes 4
6. Creación y modificación de exámenes 5
8. Mejoras en la interfaz de usuario 8
9. Validación de datos 10
10. Seguridad y control de acceso 10
12. Ejecución de la aplicación 12
13. Estado final del proyecto 12

1. Introducción

En esta última iteración se ha trabajado principalmente en completar la gestión de exámenes, incorporando las opciones de crear, consultar, modificar y eliminar exámenes. También se han realizado mejoras visuales en las pantallas de tareas, asignaturas y exámenes, con el objetivo de conseguir una interfaz más uniforme y sencilla de utilizar.

A lo largo de este documento se explica la estructura de las nuevas funcionalidades desarrolladas, las clases y métodos utilizados, las modificaciones realizadas en la interfaz y las comprobaciones llevadas a cabo para verificar su funcionamiento.

Por último, se incluyen los pasos necesarios para ejecutar la aplicación y comprobar las funcionalidades incorporadas.

2. Entidad Examen

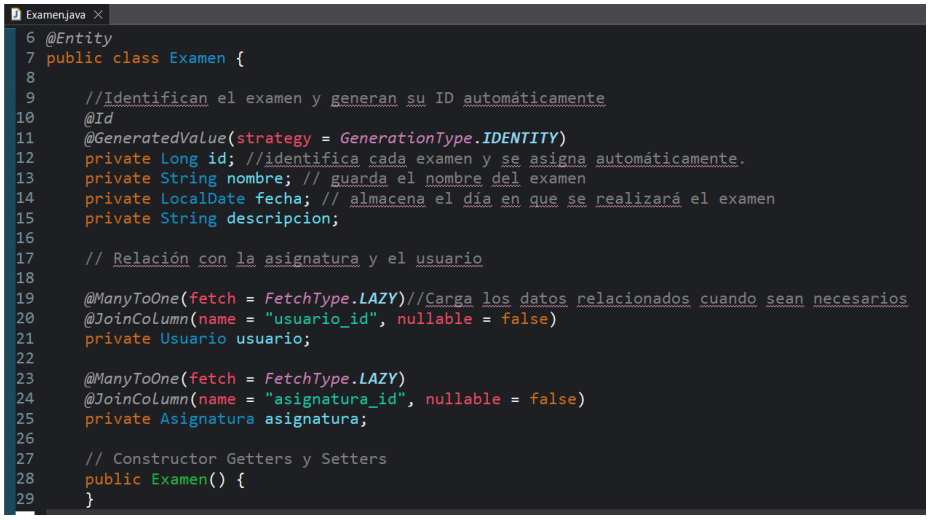
La clase **Examen.java**, situada en el **paquete entity**, representa los exámenes que los usuarios pueden gestionar en **CatchTask**. Esta clase define los datos que se almacenan en la base de datos para cada examen.

Los **atributos** principales de esta clase son **id, nombre, fecha y descripcion**. El atributo id permite distinguir cada examen y se genera automáticamente. El nombre contiene el título del examen, fecha guarda el día en el que se realizará mediante el tipo LocalDate y descripcion permite añadir información adicional sobre el examen, aunque este campo es opcional.

Para relacionar la entidad con la base de datos se utilizan varias anotaciones de JPA. @Entity indica que la clase representa una entidad que se almacena en la base de datos, @Id identifica la clave primaria y @GeneratedValue indica que el identificador de cada examen se asigna automáticamente, sin que el usuario tenga que introducirlo manualmente.

Además, la clase utiliza @ManyToOne y @JoinColumn para establecer relaciones con las entidades Usuario y Asignatura. Cada examen está asociado a un usuario y a una asignatura. Las columnas usuario_id y asignatura_id permiten guardar estas relaciones en la base de datos y, al tener configurado nullable = false, no permiten almacenar un examen sin el usuario o la asignatura correspondientes.

Por último, se utiliza FetchType.LAZY para que la información del usuario y de la asignatura se cargue solo cuando sea necesaria, evitando cargar datos que no se van a utilizar.



```java
@Entity
public class Examen {

    //Identifican el examen y generan su ID automáticamente
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; //identifica cada examen y se asigna automáticamente.
    private String nombre; // guarda el nombre del examen
    private LocalDate fecha; // almacena el día en que se realizará el examen
    private String descripcion;

    // Relación con la asignatura y el usuario

    @ManyToOne(fetch = FetchType.LAZY)//Carga los datos relacionados cuando sean necesarios
    @JoinColumn(name = "usuario_id", nullable = false)
    private Usuario usuario;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "asignatura_id", nullable = false)
    private Asignatura asignatura;

    // Constructor Getters y Setters
    public Examen() {
    }
```

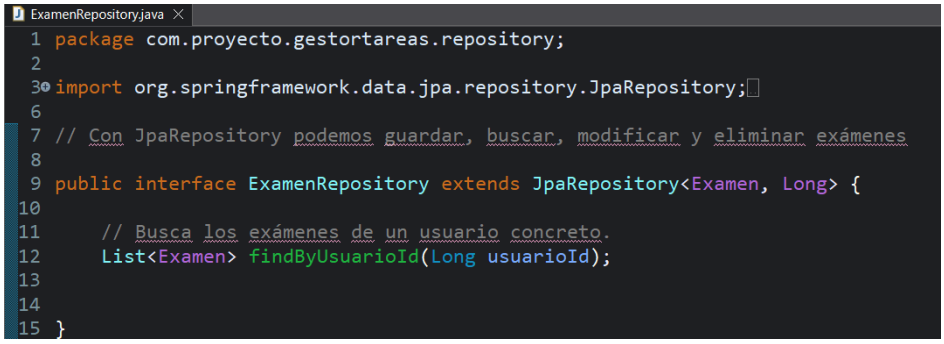
Figura 1. Código de la entidad Examen

3. Repositorio de Exámenes

La **interfaz ExamenRepository.java**, situada en el **paquete repository**, se encarga de realizar las operaciones necesarias para trabajar con los exámenes almacenados en la base de datos.

Esta interfaz extiende de JpaRepository, indicando como primer parámetro la entidad Examen y como segundo el tipo de dato de su identificador, que en este caso es Long. Gracias a esto, se pueden realizar operaciones como guardar, buscar, modificar y eliminar exámenes sin necesidad de escribir estas consultas manualmente.

Además, se ha creado el método findByUsuarioId (Long usuarioId), que permite buscar todos los exámenes que pertenecen a un usuario concreto. De esta forma, cada usuario puede consultar únicamente los exámenes que ha registrado en **CatchTask**.



```java
package com.proyecto.gestortareas.repository;

import org.springframework.data.jpa.repository.JpaRepository;

// Con JpaRepository podemos guardar, buscar, modificar y eliminar exámenes

public interface ExamenRepository extends JpaRepository<Examen, Long> {

    // Busca los exámenes de un usuario concreto.
    List<Examen> findByUsuarioId(Long usuarioId);


}
```

Figura 2. Repositorio de exámenes

4. Controlador de Exámenes

La **clase ExamenController.java**, situada en el **paquete controller**, se encarga de gestionar las diferentes operaciones relacionadas con los exámenes dentro de CatchTask. Desde este controlador se reciben las peticiones del usuario y se decide qué acciones realizar y qué páginas mostrar.

Para indicar que esta clase funciona como controlador de Spring MVC se utiliza la anotación @Controller. Además, @RequestMapping("/examenes") establece la dirección principal desde la que se gestionan todas las operaciones relacionadas con los exámenes.

El controlador utiliza dos repositorios para acceder a la información almacenada en la base de datos. Por un lado, ExamenRepository que permite buscar, guardar, modificar y eliminar exámenes. Por otro lado, AsignaturaRepository que permite consultar las asignaturas del usuario y comprobar cuál se ha seleccionado al crear o modificar un examen.

Para utilizar estos repositorios dentro del controlador se utiliza la anotación @Autowired. Esta permite que Spring se encargue de crear y proporcionar los repositorios automáticamente, sin tener que crearlos manualmente.

De esta forma, ExamenController conecta las peticiones realizadas desde las diferentes pantallas con las operaciones necesarias para gestionar los exámenes.

ExamenController.java

```
1  package com.proyecto.gestortareas.controller;
2
3⊕ import org.springframework.stereotype.Controller;
19
20 @Controller
21 @RequestMapping("/examenes")
22 public class ExamenController {
23
24     // Repositorio para acceder a los exámenes guardados.
25     @Autowired
26     private ExamenRepository examenRepository;
27
28     // Repositorio para acceder a las asignaturas.
29     @Autowired
30     private AsignaturaRepository asignaturaRepository;
```

Figura 3. Declaración del controlador de exámenes y sus repositorios.

5. Consulta de exámenes

Para mostrar los exámenes registrados en CatchTask se utiliza el **método listarExamenes()**, definido en la clase ExamenController.java.

Este método utiliza la anotación @GetMapping, que permite acceder a él mediante una petición GET a la dirección /examenes.

En primer lugar, se comprueba si existe un usuario conectado mediante la sesión. Si no hay ningún usuario, se redirige a la pantalla de inicio de sesión para impedir que se pueda acceder al listado sin haber iniciado sesión.

A continuación, se utiliza el método findByUsuarioId() del repositorio ExamenRepository para buscar los exámenes asociados al usuario conectado. De esta forma, cada usuario puede consultar únicamente sus propios exámenes.

Una vez obtenida la lista, se utiliza model.addAttribute("examenes", examenes) para enviar la información a la plantilla HTML. Finalmente, el método devuelve la vista examenes, donde se muestran los exámenes guardados con sus datos correspondientes.

```java
// Muestra los exámenes del usuario que ha iniciado sesión.
@GetMapping
public String listarExamenes(HttpSession session, Model model) {

    // Comprobamos si hay un usuario conectado.
    Usuario usuario = (Usuario) session.getAttribute("usuario");

    // Si no hay ningún usuario, volvemos al inicio de sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Buscamos los exámenes asociados a ese usuario.
    List<Examen> examenes = examenRepository.findByUsuarioId(usuario.getId());

    // Enviamos la lista de exámenes a la página HTML.
    model.addAttribute("examenes", examenes);

    return "examenes";
}
```

Figura 4. Método listarExamenes() del controlador de exámenes.

6. Creación y modificación de exámenes

Para crear y modificar exámenes se han desarrollado cuatro métodos en la clase ExamenController.java.

El método **nuevoExamen()** muestra el formulario para añadir un examen nuevo. Para ello, prepara un examen vacío y carga las asignaturas del usuario para que pueda seleccionar una.

El método **guardarExamen()** recoge los datos introducidos en el formulario, comprueba que se haya seleccionado una asignatura válida y guarda el examen en la base de datos. Después, vuelve a la pantalla de Mis exámenes mostrando un mensaje de confirmación.

Para modificar un examen se utilizan los métodos **editarExamen()** y **actualizarExamen().** El primero carga los datos del examen seleccionado y los muestra en el formulario para poder editarlos. El segundo recoge los cambios realizados y los guarda en la base de datos.

De esta forma, el usuario puede añadir nuevos exámenes y modificar los que ya tiene registrados.

```java
// Muestra el formulario para crear un nuevo examen.
@GetMapping("/nuevo")
public String nuevoExamen(HttpSession session, Model model) {

    // Comprobamos si hay un usuario conectado.
    Usuario usuario = (Usuario) session.getAttribute("usuario");

    // Si no hay ningún usuario, volvemos al inicio de sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Creamos un examen vacío para rellenar el formulario.
    model.addAttribute("examen", new Examen());

    // Buscamos las asignaturas del usuario conectado.
    List<Asignatura> asignaturas = asignaturaRepository.findByUsuarioId(usuario.getId());

    // Enviamos las asignaturas al formulario.
    model.addAttribute("asignaturas", asignaturas);

    return "formulario-examen";

}
```

Figura 5.Método para mostrar el formulario

```java
@PostMapping("/guardar")
public String guardarExamen(

    @ModelAttribute("examen") Examen examen,
    HttpSession session,
    RedirectAttributes redirectAttributes) {

    // Obtiene el usuario conectado.
    Usuario usuario = (Usuario) session.getAttribute("usuario");

    // Impide guardar exámenes sin iniciar sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Comprueba que se haya seleccionado una asignatura.
    if (examen.getAsignatura() == null || examen.getAsignatura().getId() == null) {

        redirectAttributes.addFlashAttribute(
                "error", "Debes seleccionar una asignatura.");

        return "redirect:/examenes/nuevo";
    }

    // Busca la asignatura seleccionada en la base de datos.
    Asignatura asignatura = asignaturaRepository
            .findById(examen.getAsignatura().getId())
            .orElse(null);

    // Comprueba que la asignatura exista y pertenezca al usuario.
    if (asignatura == null || !asignatura.getUsuario().getId().equals(usuario.getId())) {

        redirectAttributes.addFlashAttribute(
                "error", "La asignatura seleccionada no es válida.");

        return "redirect:/examenes/nuevo";
    }

    // Asocia el examen al usuario conectado.
    examen.setUsuario(usuario);

    // Asocia el examen a la asignatura seleccionada.
    examen.setAsignatura(asignatura);

    // Guarda el examen en la base de datos.
```

Figura 6. Método para guardar un nuevo examen



```
public String editarExamen(
        @PathVariable Long id,
        HttpSession session,
        Model model,
        RedirectAttributes redirectAttributes) {

    // Comprobamos si hay un usuario conectado.
    Usuario usuario = (Usuario) session.getAttribute("usuario");

    if (usuario == null) {
        return "redirect:/login";
    }

    // Buscamos el examen seleccionado.
    Examen examen = examenRepository.findById(id).orElse(null);

    // Comprobamos que el examen exista y pertenezca al usuario.
    if (examen == null || !examen.getUsuario().getId().equals(usuario.getId())) {

        redirectAttributes.addFlashAttribute(
                "error", "No se ha encontrado el examen.");

        return "redirect:/examenes";

    }

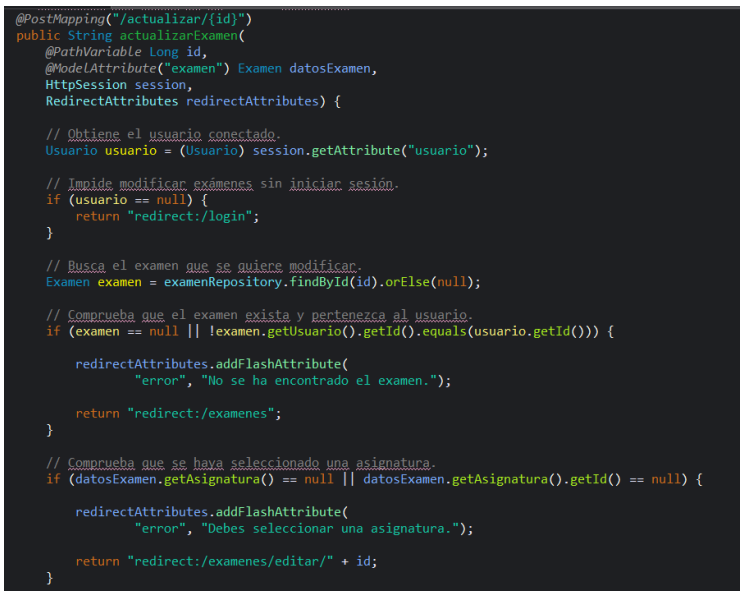
    // Enviamos el examen al formulario.
    model.addAttribute("examen", examen);

    // Buscamos las asignaturas del usuario conectado.
    List<Asignatura> asignaturas = asignaturaRepository.findByUsuarioId(usuario.getId());

    // Enviamos las asignaturas al formulario.
    model.addAttribute("asignaturas", asignaturas);

    return "formulario-examen";
}
```

Figura 7. Método para cargar los datos de un examen antes de modificarlo.



```
@PostMapping("/actualizar/{id}")
public String actualizarExamen(
    @PathVariable Long id,
    @ModelAttribute("examen") Examen datosExamen,
    HttpSession session,
    RedirectAttributes redirectAttributes) {

    // Obtiene el usuario conectado.
    Usuario usuario = (Usuario) session.getAttribute("usuario");

    // Impide modificar exámenes sin iniciar sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Busca el examen que se quiere modificar.
    Examen examen = examenRepository.findById(id).orElse(null);

    // Comprueba que el examen exista y pertenezca al usuario.
    if (examen == null || !examen.getUsuario().getId().equals(usuario.getId())) {

        redirectAttributes.addFlashAttribute(
                "error", "No se ha encontrado el examen.");

        return "redirect:/examenes";
    }

    // Comprueba que se haya seleccionado una asignatura.
    if (datosExamen.getAsignatura() == null || datosExamen.getAsignatura().getId() == null) {

        redirectAttributes.addFlashAttribute(
                "error", "Debes seleccionar una asignatura.");

        return "redirect:/examenes/editar/" + id;
    }
```

Figura 8. Mitad método para actualizar un examen existente.

7. Eliminación de exámenes

Para permitir que el usuario pueda eliminar los exámenes que ya no necesita, se ha desarrollado el **método eliminarExamen()** en la clase ExamenController.java.

Este método recibe el identificador del examen que se quiere eliminar y comprueba que el usuario haya iniciado sesión y que el examen le pertenezca.

Si las comprobaciones son correctas, se utiliza el **método delete()** del repositorio ExamenRepository para eliminar el examen de la base de datos. Finalmente, se muestra un mensaje de confirmación y se vuelve a la pantalla de Mis exámenes.

```java
@PostMapping("/eliminar/{id}")
public String eliminarExamen(
    @PathVariable Long id,
    HttpSession session,
    RedirectAttributes redirectAttributes) {

    // Obtiene el usuario conectado.
    Usuario usuario = (Usuario) session.getAttribute("usuario");

    // Impide eliminar exámenes sin iniciar sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Busca el examen que se quiere eliminar.
    Examen examen = examenRepository.findById(id).orElse(null);

    // Comprueba que el examen exista y pertenezca al usuario.
    if (examen == null || !examen.getUsuario().getId().equals(usuario.getId())) {

        redirectAttributes.addFlashAttribute(
                "error", "No se ha encontrado el examen.");

        return "redirect:/examenes";
    }

    // Elimina el examen de la base de datos.
    examenRepository.delete(examen);

    // Muestra un mensaje de confirmación.
    redirectAttributes.addFlashAttribute(
            "mensaje", "Examen eliminado correctamente.");

    // Vuelve al listado de exámenes.
    return "redirect:/examenes";
}
```

Figura 9. Método para eliminar un examen de CatchTask.

8. Mejoras en la interfaz de usuario

Durante este sprint también se han realizado algunos cambios en el diseño de **CatchTask** para conseguir que las diferentes pantallas tengan una apariencia más uniforme y ordenada.

En la pantalla de tareas se han ajustado algunos elementos para facilitar la lectura de la información. En la pantalla de asignaturas se han modificado los espacios entre los datos y la distribución de los botones.

Por otro lado, se ha adaptado el diseño de la pantalla de exámenes para que siga el mismo estilo que las pantallas anteriores, utilizando tarjetas para mostrar la información de cada examen.

Además, se han añadido frases motivadoras y enlaces para cerrar sesión en las diferentes pantallas, mejorando así la presentación de la aplicación y facilitando su uso.

Con estos cambios se ha conseguido mantener un diseño más sencillo y parecido en las principales secciones de **CatchTask**.

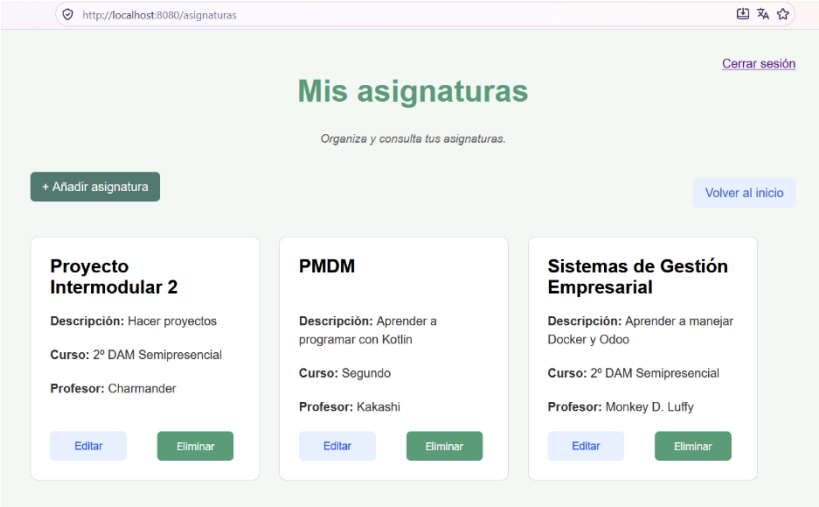


Figura 10. Pantalla de Mis asignaturas.

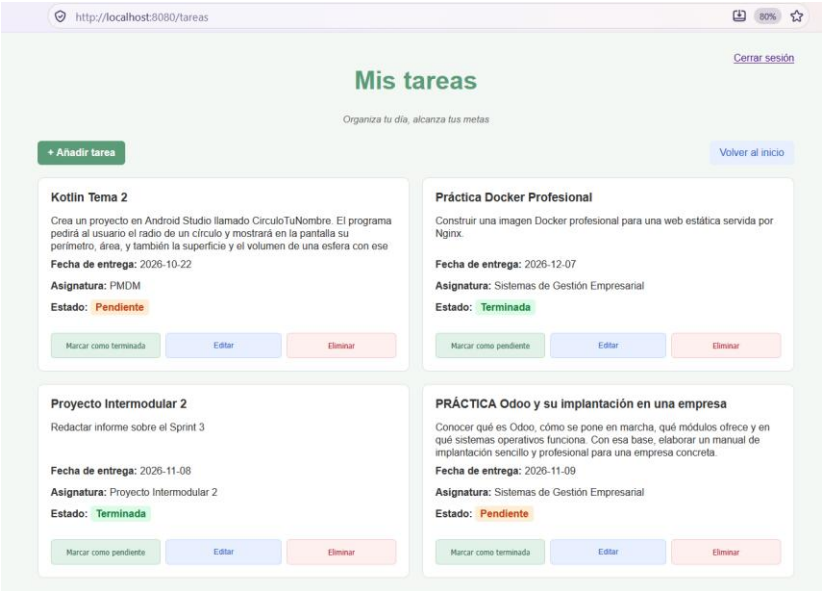


Figura 11. Pantalla de Mis tareas.

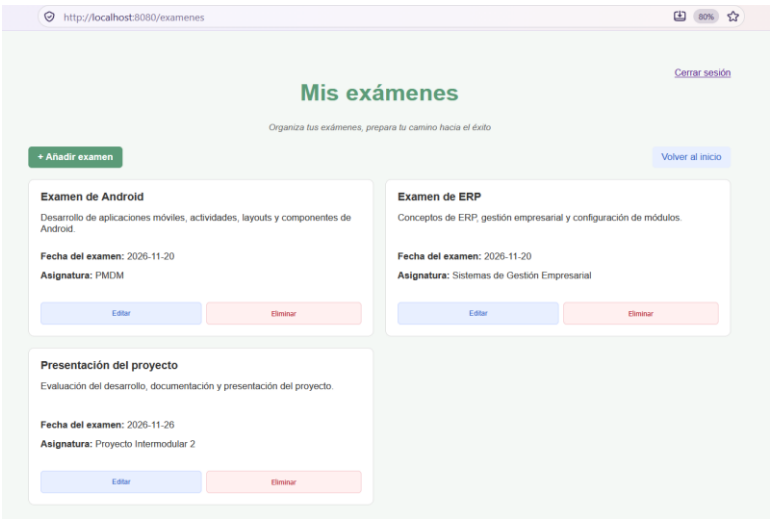


Figura 12. Pantalla de Mis exámenes.

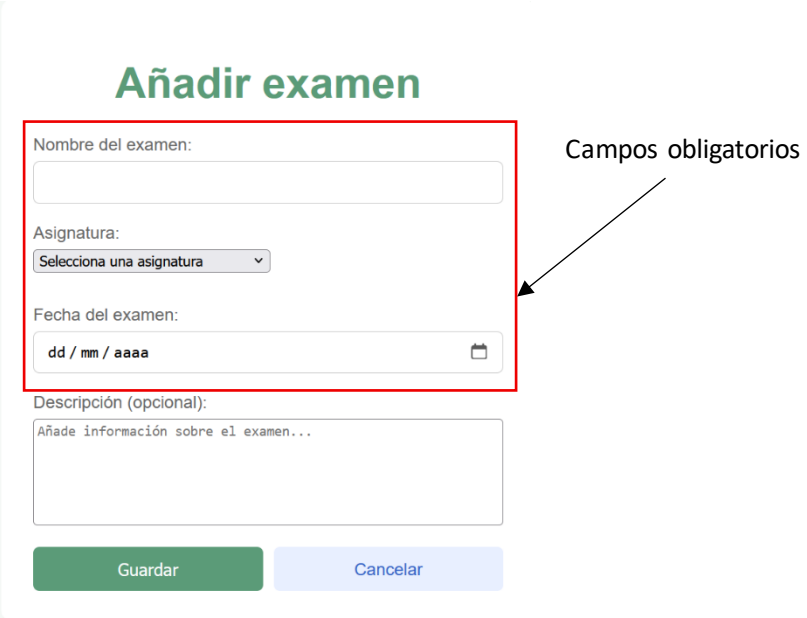
9. Validación de datos

Durante el desarrollo de la gestión de exámenes se han añadido algunas comprobaciones para evitar que se guarden datos incorrectos.

En primer lugar, el formulario obliga a introducir los campos necesarios, como el nombre y la fecha del examen, y a seleccionar una asignatura. La descripción, en cambio, es opcional.

Además, desde el controlador se comprueba que la asignatura seleccionada exista y pertenezca al usuario que está utilizando la aplicación. De esta forma, se evita asociar un examen a una asignatura de otro usuario.

Estas comprobaciones permiten guardar los exámenes con la información necesaria y evitar algunos errores durante su creación y modificación.



10. Seguridad y control de acceso

Durante el desarrollo de la gestión de exámenes se han mantenido las comprobaciones de seguridad utilizadas en el resto de la aplicación.

En los diferentes métodos del controlador se comprueba que el usuario haya iniciado sesión. Si no existe un usuario conectado, se le redirige a la pantalla de inicio de sesión.

Además, antes de modificar o eliminar un examen, se comprueba que pertenezca al usuario que está realizando la operación. También se verifica que la asignatura seleccionada sea del mismo usuario.

De esta forma, cada usuario puede gestionar sus propios exámenes sin acceder a la información de otras personas.

```
// Obtiene el usuario conectado.
Usuario usuario = (Usuario) session.getAttribute("usuario");

// Impide eliminar exámenes sin iniciar sesión.
if (usuario == null) {
    return "redirect:/login";
}

// Busca el examen que se quiere eliminar.
Examen examen = examenRepository.findById(id).orElse(null);

// Comprueba que el examen exista y pertenezca al usuario.
if (examen == null || !examen.getUsuario().getId().equals(usuario.getId())) {

    redirectAttributes.addFlashAttribute(
            "error", "No se ha encontrado el examen.");

    return "redirect:/examenes";
}
```

Figura 14. Comprobaciones de seguridad para eliminar un examen.

11. Pruebas y verificación de la Definition of Done

Una vez finalizado el desarrollo de las funcionalidades del Sprint 4, se realizaron diferentes pruebas para comprobar que la gestión de exámenes funcionaba correctamente y cumplía los criterios de aceptación establecidos en las Historias de Usuario.

Se comprobó la creación de nuevos exámenes, verificando que los datos introducidos se guardaran correctamente y que aparecieran en el listado del usuario. También se probaron las opciones de modificación y eliminación, comprobando que los cambios se guardaran correctamente.

Además, se revisaron las validaciones de los campos obligatorios y las comprobaciones de seguridad para evitar que un usuario pudiera gestionar exámenes o utilizar asignaturas pertenecientes a otras cuentas.

Estas comprobaciones permitieron verificar que las funcionalidades desarrolladas durante el Sprint 4 estaban implementadas y listas para su evaluación, sin errores bloqueantes conocidos.

Prueba realizada	Resultado esperado	
Crear un examen	Se guarda y aparece en el listado.	**Correcto**
Consultar los exámenes	Se muestran los exámenes del usuario conectado.	**Correcto**
Modificar un examen	Se actualizan sus datos correctamente.	**Correcto**
Eliminar un examen	Desaparece del listado después de confirmar.	**Correcto**
Validar campos obligatorios	No se permite enviar el formulario incompleto.	**Correcto**
Comprobar el acceso a exámenes de otro usuario	Se impide acceder o modificar exámenes ajenos.	**Correcto**
Comprobar la propiedad de las asignaturas	No se permite utilizar asignaturas de otras cuentas.	**Correcto**

12. Ejecución de la aplicación

Actualmente, **CatchTask** se ejecuta de forma local, ya que todavía no se ha realizado su despliegue en un servidor en la nube. El código fuente del proyecto se encuentra alojado en GitHub, desde donde se puede obtener la versión desarrollada.

Para ejecutar la aplicación, es necesario disponer de Java 21, Eclipse, Maven y MySQL Workbench, además de tener configurada la base de datos utilizada por el proyecto.

Los pasos para poner en marcha el proyecto son los siguientes:

1. Descargar o clonar el repositorio de GitHub en el equipo.

2. Crear la base de datos MySQL con el nombre **gestortareas**.

3. Configurar las credenciales de conexión a MySQL mediante las propiedades del archivo application.properties, incluyendo la variable de entorno DB_PASSWORD.

4. Comprobar que el servidor MySQL está iniciado.

5. Abrir el proyecto desde el entorno de desarrollo y ejecutar la clase principal de Spring Boot o iniciar el proyecto mediante Maven.

6. Al iniciar la aplicación, Hibernate se encarga de crear o actualizar automáticamente las tablas necesarias, gracias a la propiedad spring.jpa.hibernate.ddl-auto=update.

7. Abrir el navegador y acceder a **http://localhost:8080/login** para utilizar CatchTask.

Una vez iniciada la aplicación, se puede acceder a las diferentes secciones y comprobar las funcionalidades desarrolladas durante el Sprint 4, como la creación, modificación, eliminación y consulta de exámenes, además de las mejoras visuales realizadas en las pantallas de tareas, asignaturas y exámenes.

13. Estado final del proyecto

Con la finalización del Sprint 4, se da por concluido el desarrollo de **CatchTask**. La aplicación cuenta con las funcionalidades principales para gestionar tareas, asignaturas y exámenes, permitiendo crear, consultar, modificar y eliminar estos elementos.

También se han realizado mejoras visuales para conseguir una interfaz más uniforme y sencilla de utilizar.

Actualmente, la aplicación funciona de forma local y cumple los objetivos establecidos durante los diferentes sprints. Aunque podrían incorporarse nuevas mejoras en futuras versiones, se considera finalizada la versión desarrollada para este proyecto.